Multi-robot Search and Rescue

Charvi Gupta
Rachel Zheng
Narendar Sriram
Jaekyung Song
Indraneel Patil

Sponsors: Prof. Katia Sycara, Jaskaran Grover



robo
SAR

Master of Science
Robotic Systems Development

THE ROBOTICS INSTITUTE

Meet the Team



Charvi Gupta
Path Planning Lead



Rachel Zheng
Task Allocation Lead



Narendar Sriram
Swarm Control Lead



Jaekyung Song
Localisation Lead



Indraneel Patil
Fleet Management Lead



Prof. Katia Sycara



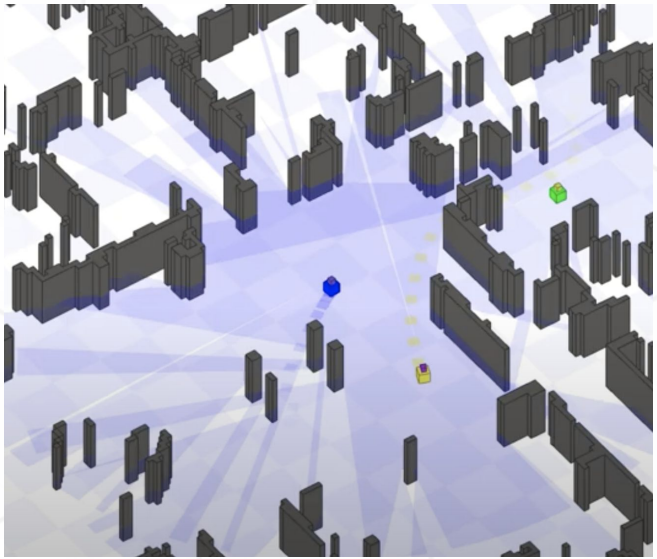
Jaskaran Grover

Project Description : Multi Robot Search and Rescue

- Search and rescue is labor intensive and dangerous so robots can help alleviate risk
- Speed is often critical so a multi robot fleet can be used to reduce search time

Salient features :

1. Coordination in search using centralised planning, task allocation and task generation
2. Information sharing on victim hotspots
3. Scalable swarm fleet based on nature of task
4. Simple robots + complex server



Use Case

There is a **dangerous natural gas leak** in a school on a working day. As the first responders work to stop the flow of gas, students inside the school must be evacuated away from the danger.

Time is critical and **all students must be located** and rescued before they are harmed by the leak.

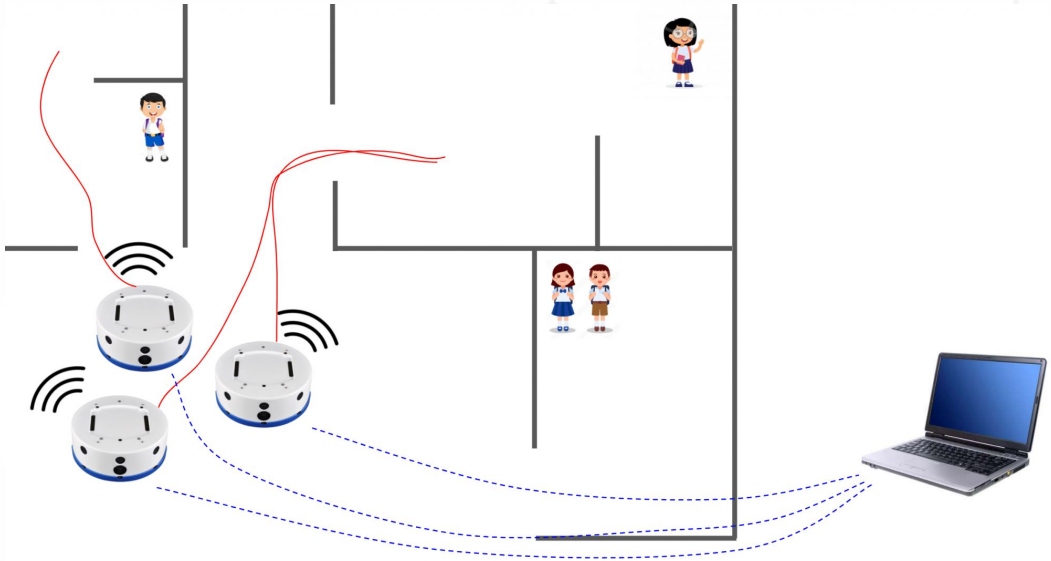


Use Case

First responders deploy **RoboSAR robot agents** to search the school for students. Agents search the entire school **whilst talking to each other and sharing information.**

As and when students are found agents **report their locations** to the first responders so that they can be rescued

Agents **return to base** after **searching every room** in the school.



System-level requirements: Mandatory Requirements

M.F.0.	**System shall identify and localize victims in environment using fiducial markers within desired rescue time**
M.F.1.	**System shall localize robots with a known map of the environment**
M.F.2.	**System shall dynamically allocate tasks to agents in the fleet for optimizing time required for the search**
M.F.3.	System shall ensure simultaneous navigation of agents which have been allocated a task
M.F.4.	System shall adapt to changes in the fleet size like addition of agents by the users or loss of robots during operation.

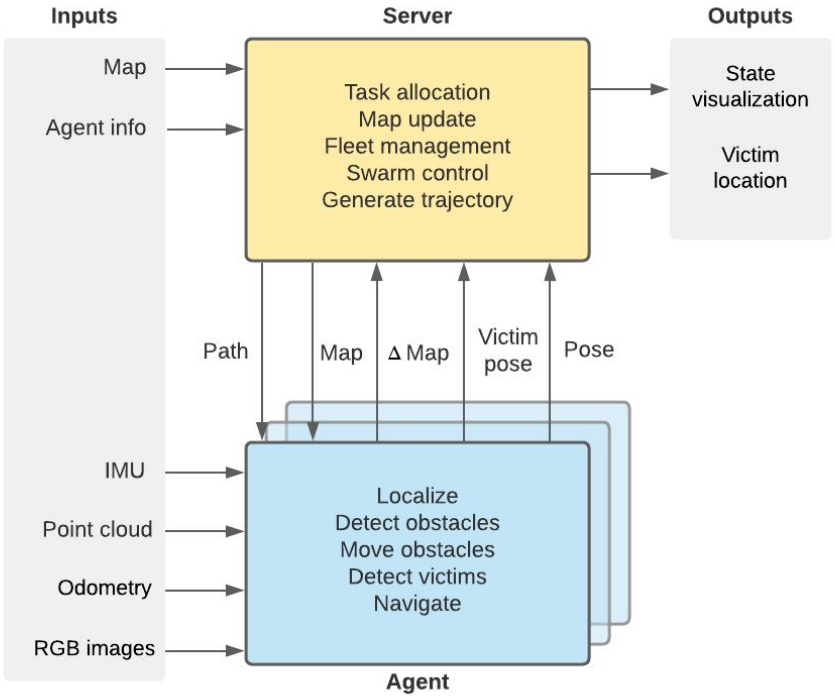
System-level requirements: Mandatory Performance Requirements

M.P.0.	**System will detect ~~95%~~ 75% of victims in the environment.**
M.P.1.	System will localize victim position within 20cm.
M.P.2.	~~Will finish searching environment 100% faster than a single robot.~~ **System will finish searching environment 20% faster than the naive algorithm for a typical indoor environment.**
M.P.3.	System will have downtime of less than 5s for busy agents.
M.P.4.	System will update agent status for user at 1Hz frequency.
M.P.5	**System will handle at least ~~4~~ 3 agents in the fleet.**

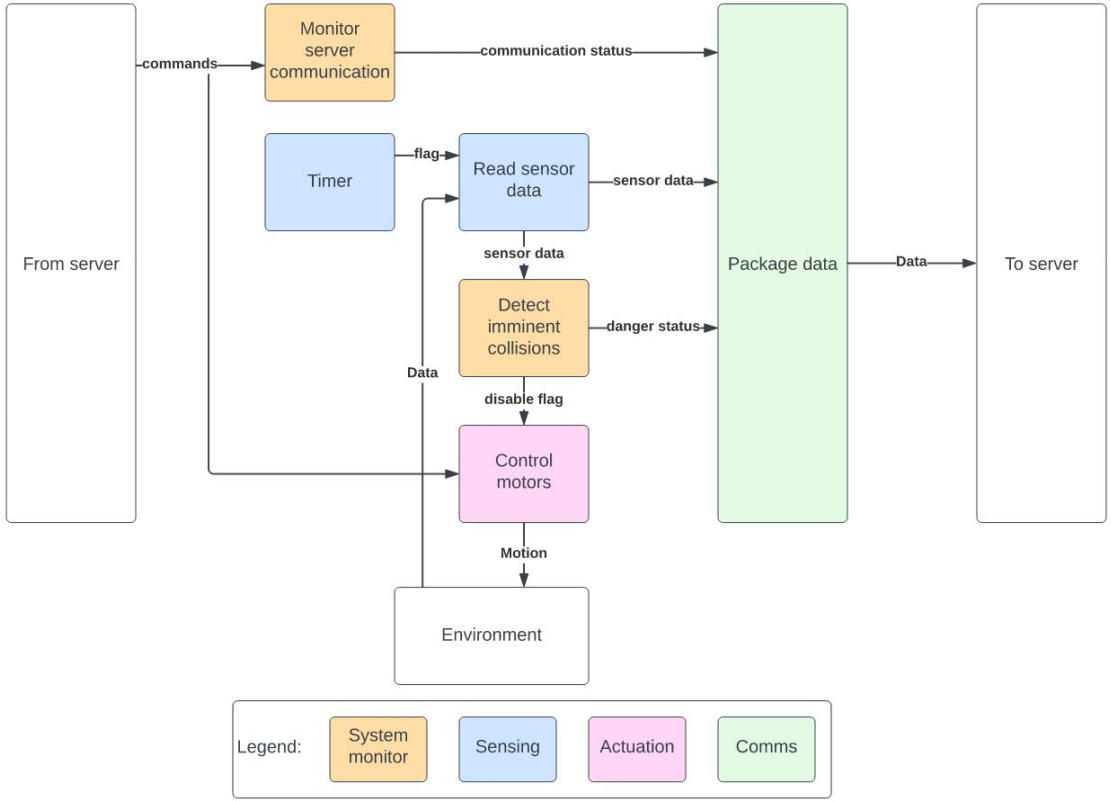
System-level requirements: Mandatory Non-Functional Requirements

M.N.0.	**System shall visualize the environment, locations of victims, and robot agents during operation**
M.N.1.	System shall home agents on emergency stops, and after completion of search
M.N.2.	**System shall have user interface to add/subtract robots during operation**
M.N.3.	System shall be well documented and have reusable software framework for swarm robotics

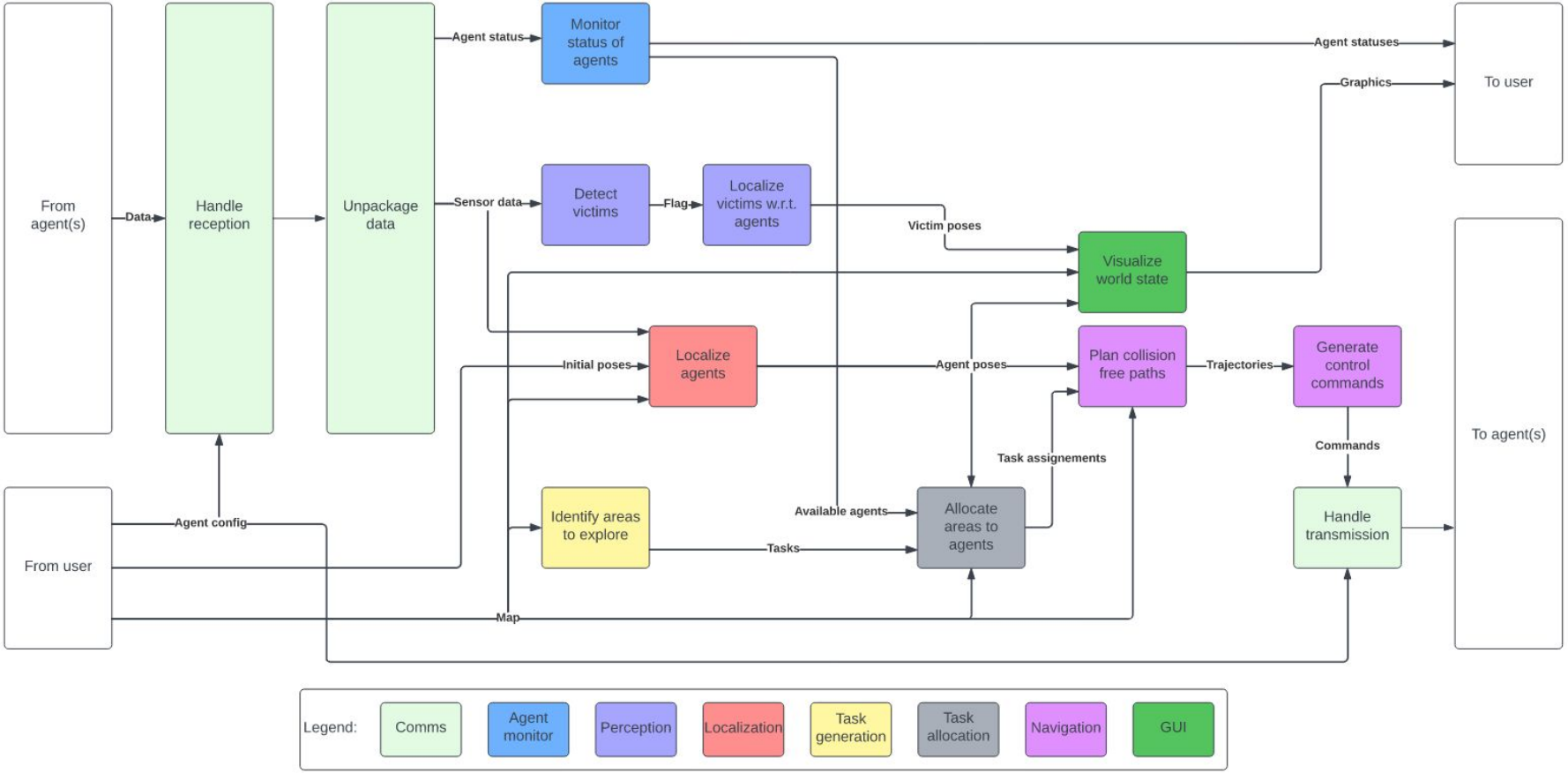
Functional architecture: RoboSAR



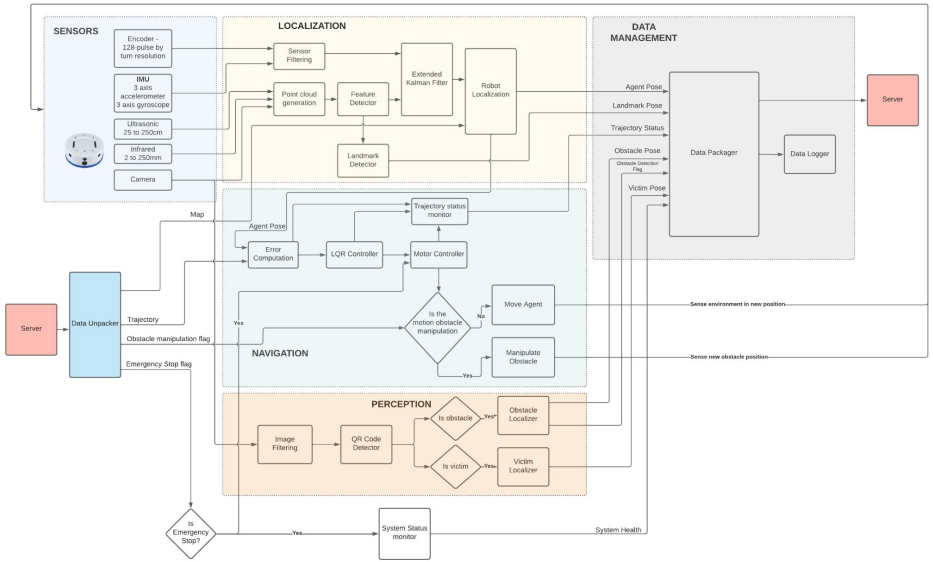
Functional architecture: Agent

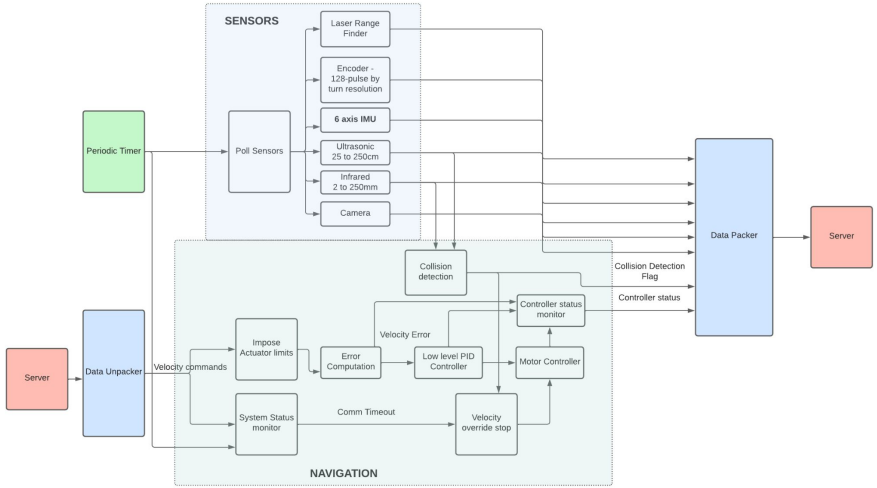


Functional architecture: Server

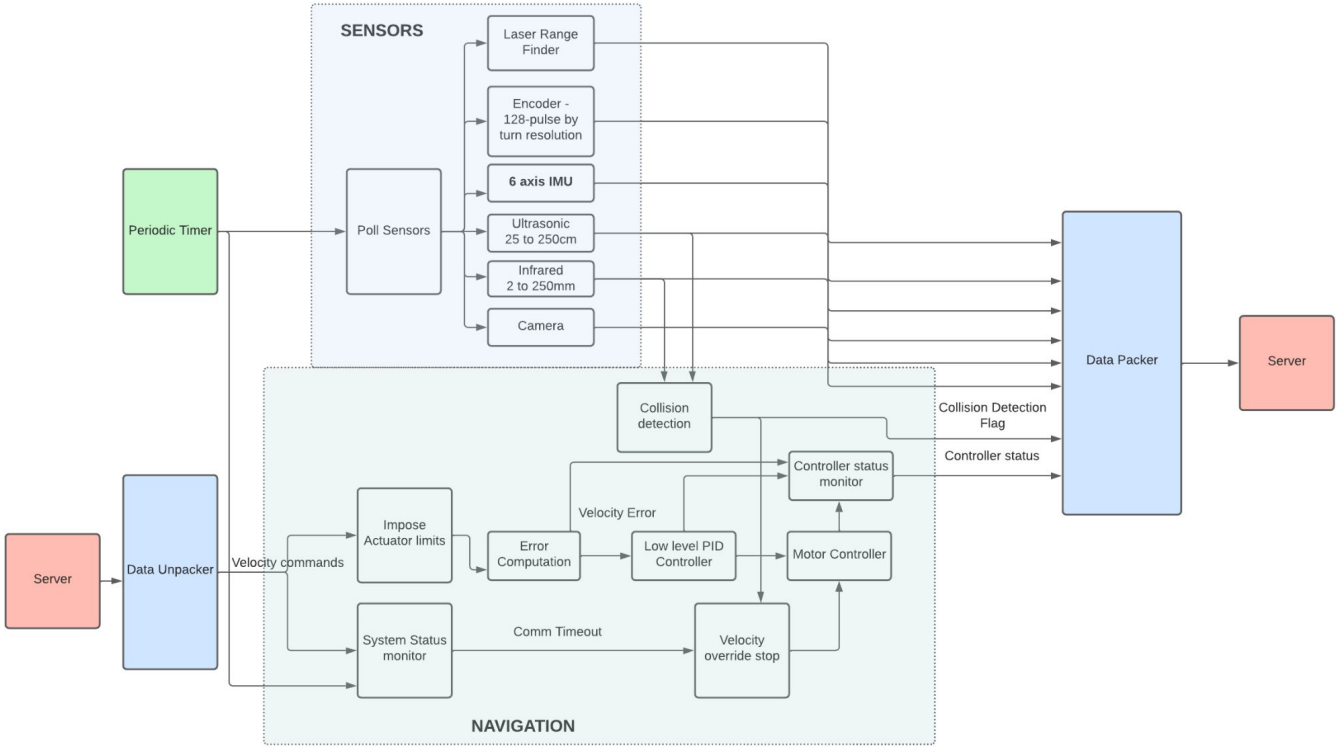


Cyberphysical architecture evolution: Agents

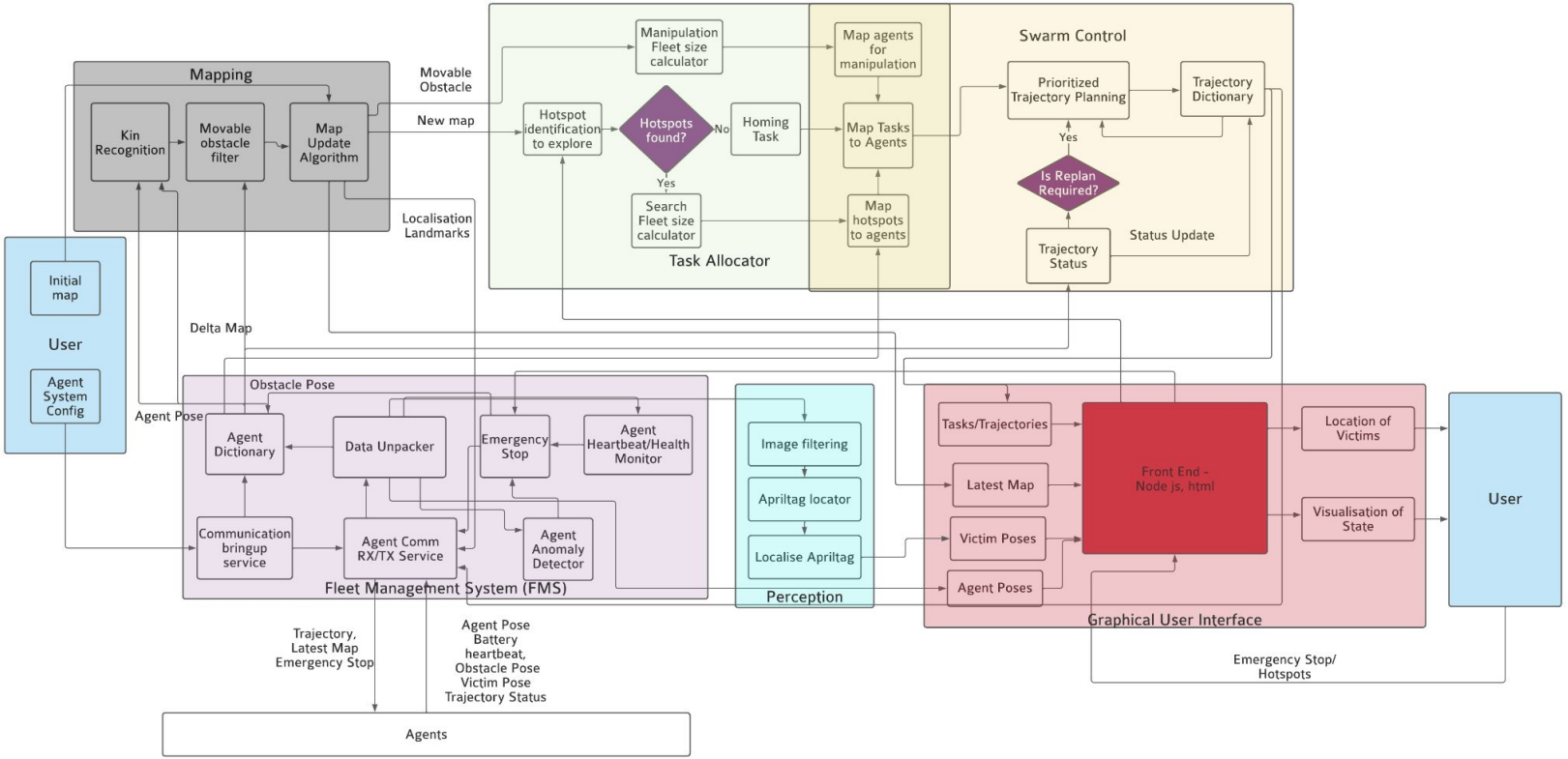




Cyberphysical architecture : Agents



Cyberphysical architecture : Server

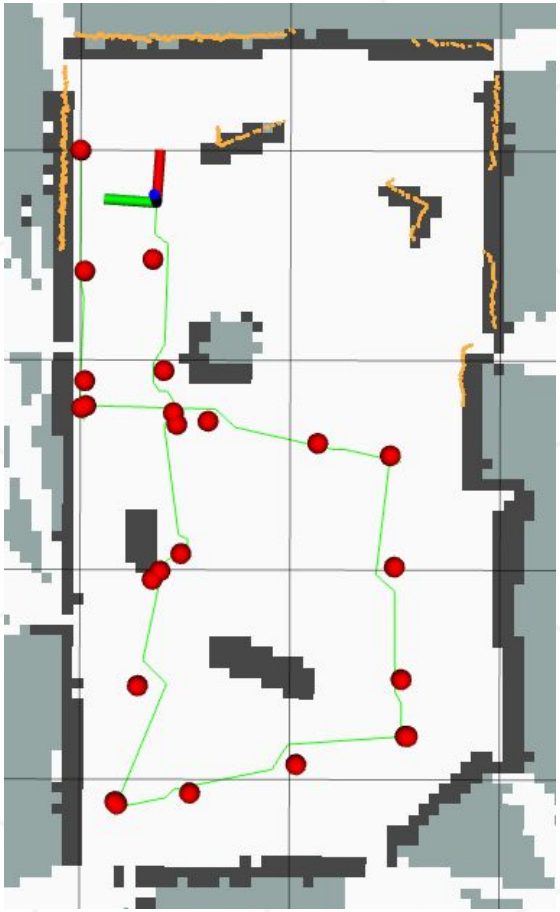


Systems Overview

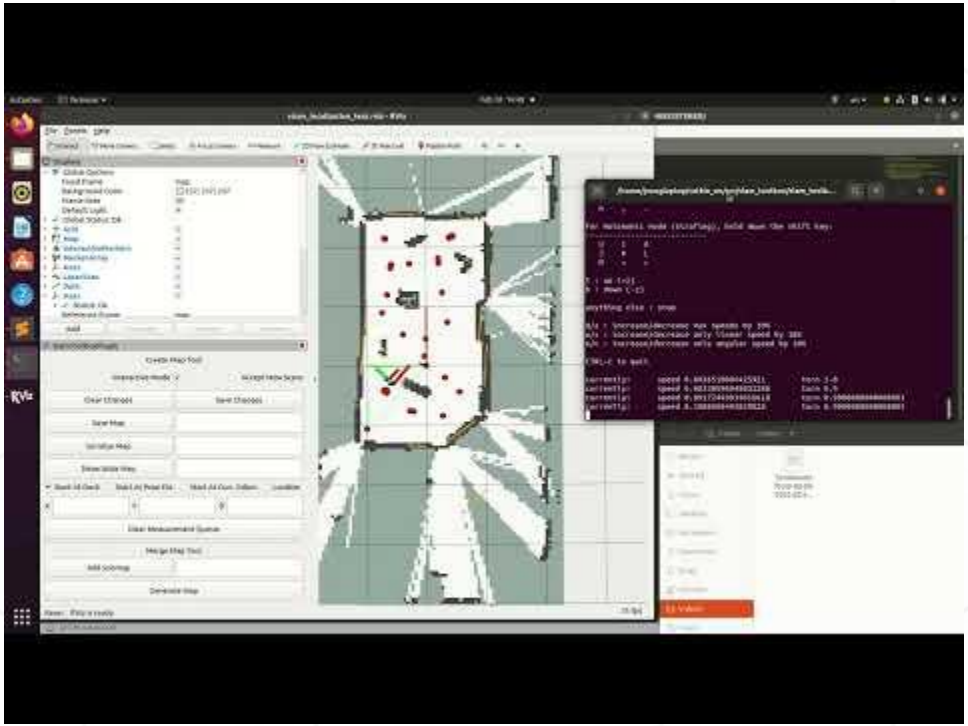
1. Localization
2. Perception
3. Task generation
4. Task allocation
5. Path planning
6. Fleet management
7. Simulation
8. Swarm control

System: Localization

- Description:
  - Determine pose of an agent given a map of the environment
- Input:
  - Initial pose
  - Occupancy grid map of environment
  - Wheel encoder values
  - Planar LIDAR range and bearing measurements
- Output:
  - Robot pose
- Process:
  - Convert wheel encoder values to odometry
  - Feed initial pose, map, odometry and LIDAR to SLAM Toolbox
  - Operate SLAM Toolbox in localization mode
  - SLAM Toolbox outputs robot pose



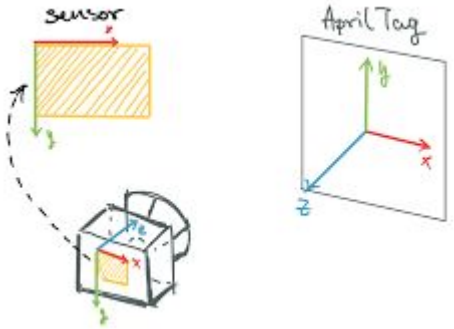
Localization Demo



Current Status: Localization

- Completed:
  - Low-level drivers for reading sensors on agents
  - Communication of sensor data from agents to server
  - Convert wheel encoder to odometry
  - Feed initial pose, odometry, LIDAR and map to SLAM Toolbox
  - Setup and utilize VICON to provide ground truth for quantifying performance
  - Localize single agent with position error of less than 10 cm RMS
- In Progress:
  - Extend localization to multiple agents
  - Perform testing on larger, more complex environments
  - Reduce position error

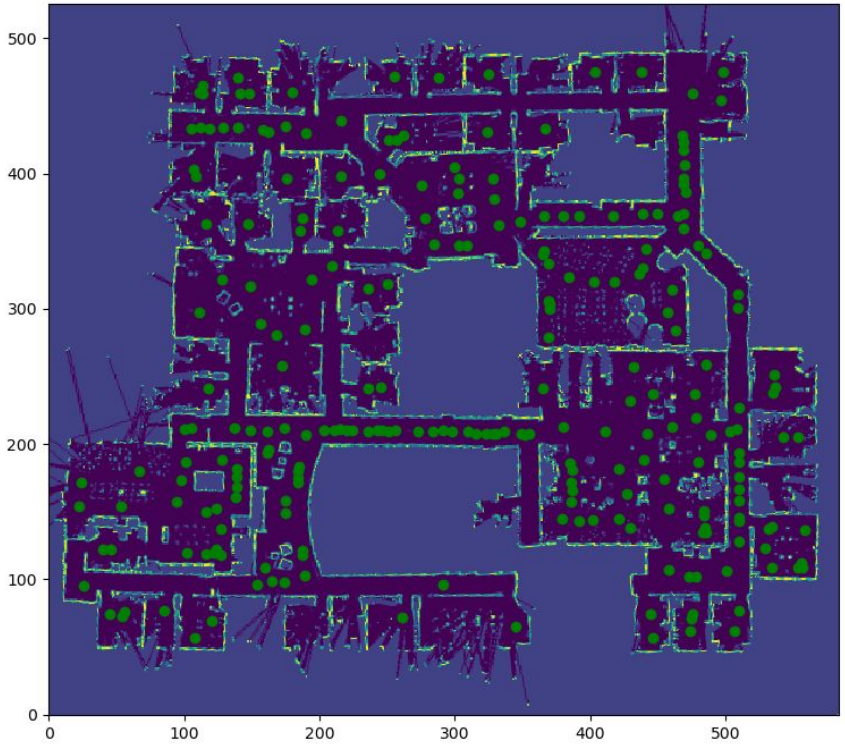
System: Perception



- Description:
  - Detect and determine the pose of a victim
- Input:
  - Camera data
- Output:
  - Victim pose
- Process:
  - Continuously monitor camera data to check for victims
  - Once victim is identified, output its pose with respect to the agent
  - Convert the relative pose to a global pose
  - Send the global victim pose to GUI
- Status:
  - Postpone development until the start of fall semester

System: Task generation

- Description:
  - Given a map, determine areas to visit so all hallways and rooms are explored
- Input:
  - Occupancy grid map of the environment
- Output:
  - List of points to visit
- Process:
  - In a map, find medium and large using randomized algorithm to find hallways and rooms
  - Iteratively try to find better points of interest, finding new or larger areas
  - Return results after final iteration



Current Status: Task generation

- Completed:
  - First draft design and implementation
  - Integration to work on ROS network
- In Progress:
  - Remove redundant or unnecessary points of interest
  - Improve coverage of map
  - Make algorithm run online / real-time

System: Task allocation

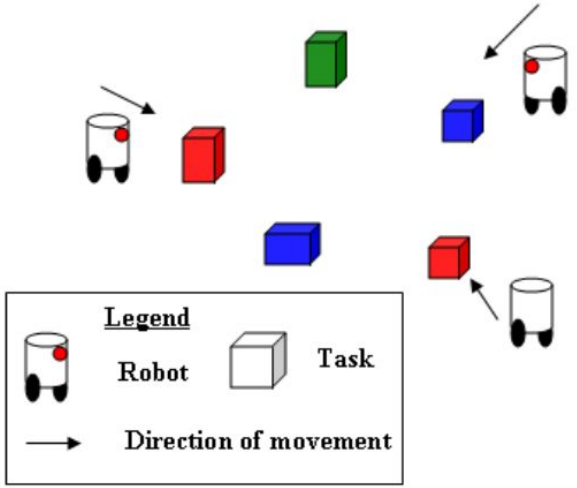
Efficiently assign each robot tasks such that all tasks are completed in minimal time.

Input:

- Map
- *n* robots
- *m* tasks
- robot initial location

Output:

- Next assigned task to robot *i*



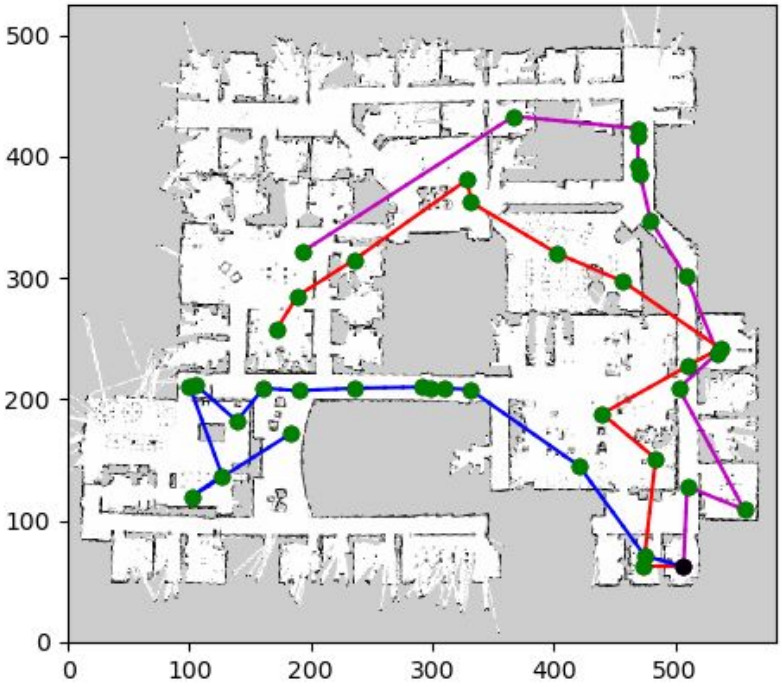
Current Status: Task allocation

Completed

- Online greedy task allocator
- Testing task allocation in Stage simulator

In Progress

- More optimal task allocation using MILP
- Testing on physical robots

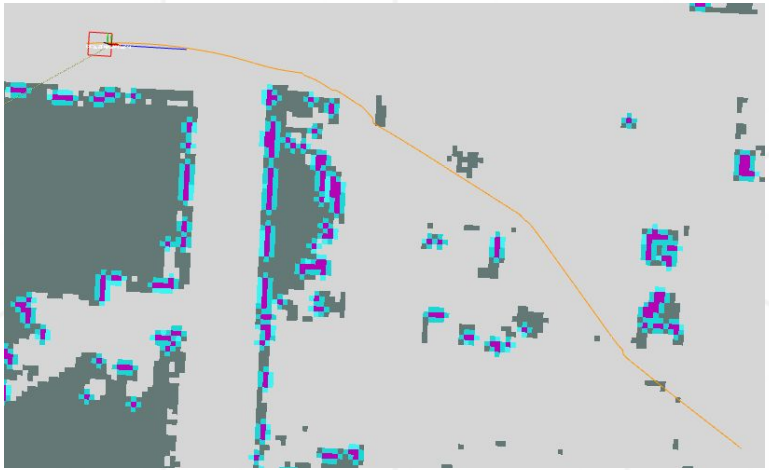


System: Path Planning

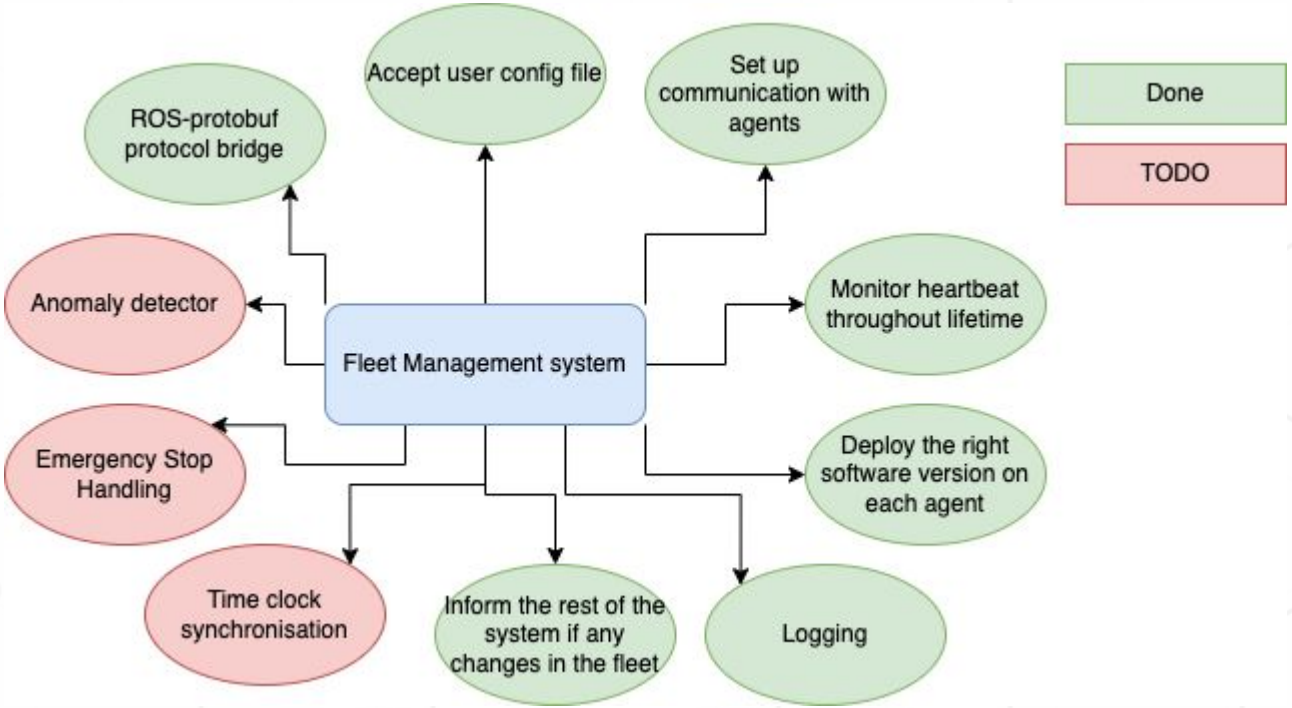
- Description:
  - Generate paths for all robots to complete their assigned tasks without collision
- Input:
  - Occupancy grid map of the environment
  - Initial location of the agents
  - Goal location of the agents
- Output:
  - Path that the agents needs to follow

Current Status: Path Planning

- Completed:
  - Integrated ros move_base for navigation
- To-do:
  - Implement a multi-robot global planner which generates collision free trajectories for all agents
  - Implement a controller for local planning



System: Fleet management

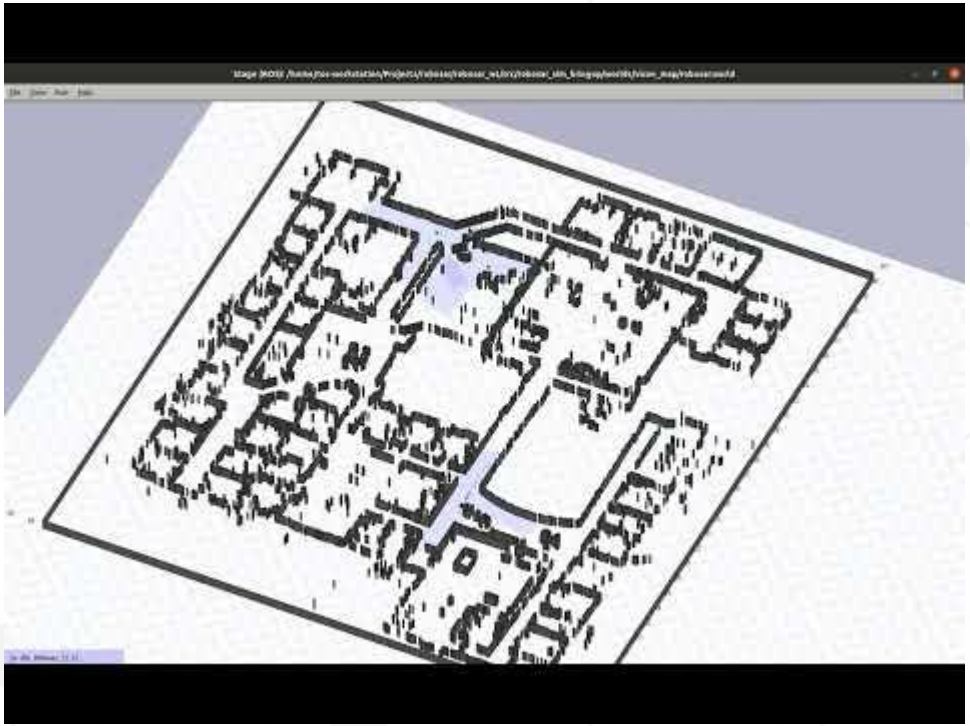


System: Simulation

Select requirements of the simulator	Selected Simulator : Stage ROS
1. **Work well with multiple robots!** 2. Should be compatible with ROS and rviz 3. Should be able to load yaml map files into the simulator easily and iterating on environments should be easy 4. Should allow simulating sensor data from robots for search and exploration	1. 2.5D simulator (can only rotate about z axis) 2. Excellent ROS interface (apart from the poor documentation) ! 3. Minimalist simulations allow simulating large populations even on our laptops 4. Drop in replacement for actual robot agents!

Current Status: Simulation

- Can simulate arbitrary shaped environment
- Can simulate arbitrary number of robots
- Can test most software subsystems on simulation before hardware testing!



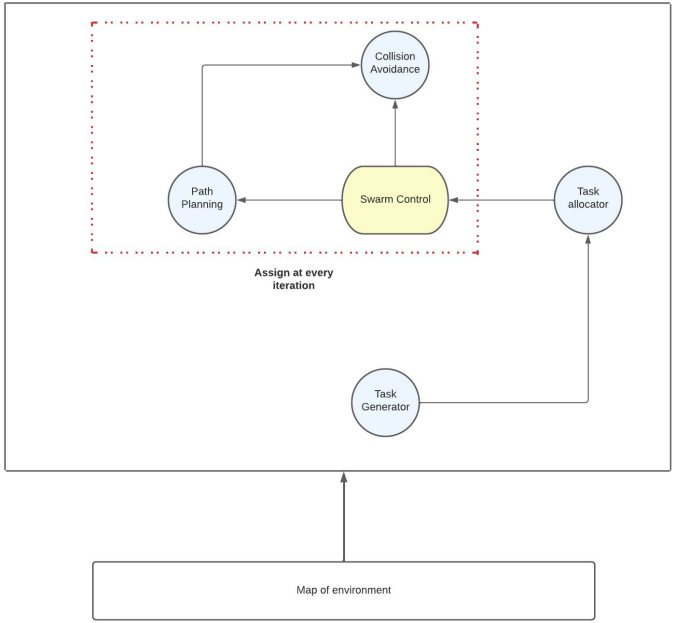
System: Swarm control

Description:
- Control formations of robots to move efficiently as a group.

Input:
- Occupancy grid map, local trajectory.

Output:
- Local trajectories for each robot.

Process:
- Given areas of interest, assign the n agents to the m tasks using Task Allocation.
- Predict agent-to-agent collision over the entire trajectory of each agent.
- Based on the prediction, come up with the most efficient multi-agent formation
- Make a motion based on the formation and repeat steps 1-3.



Current Status: Swarm Control

Since this subsystem builds upon all the other subsystems, no demonstrable work has been done it yet.

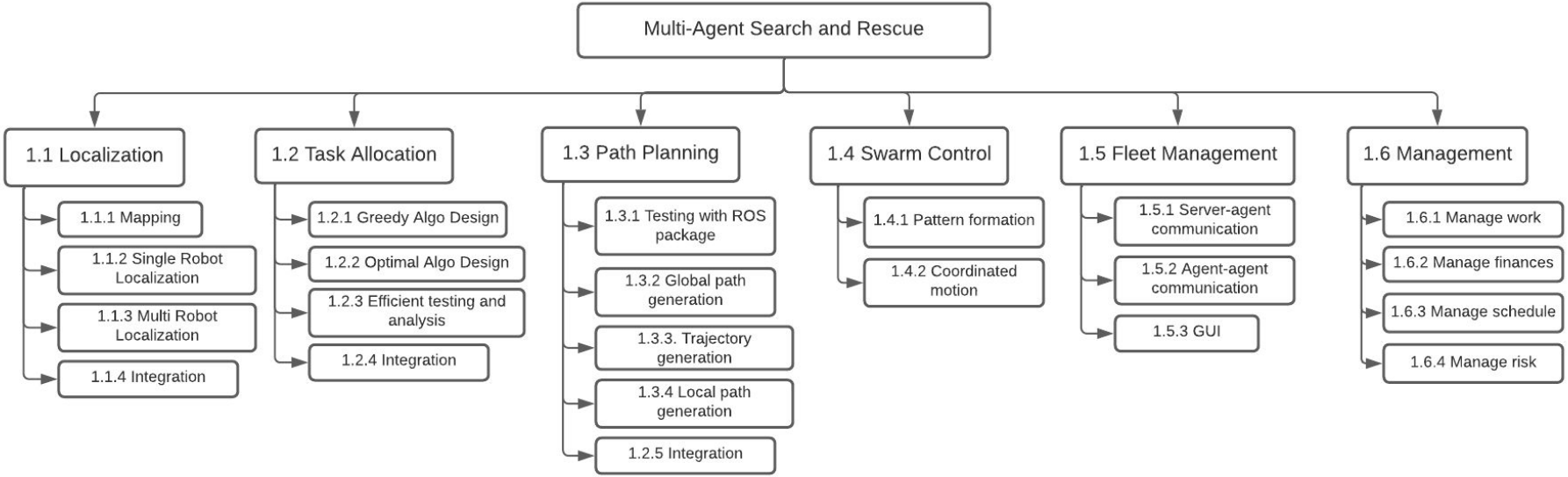
To-do:

- Literature review and trade study for best swarm control algorithms for our use case
- Test collision-free swarm control algorithm with simple formations in simulation.
- Implement working trajectory tracking feedback on top of this.
- Implement the above algorithm with on-field agents

Project Management Overview

1. WBS
2. Schedule
3. High-level test plan
4. Budget
5. Risk Management

Work Breakdown Structure



Schedule

	PR3	03/24 - 04/01	PR4	SVD
Fleet Management	Ability to communicate with and control multiple robots	Improve server-agent communication	Implement failure behavior for robots	Ability to communicate with and control multiple robots
Localization	Start designing multi-robot localization	Multi-robot localization	Improve and test multi-robot localization	Localization within 15cm accuracy
Path Planning	Custom global planner and trajectory generation	Custom local path planner	Improve and test global and local path planner	Collision-free trajectory generation Ability to successfully navigate to allocated tasks
Task Allocation	Efficient task allocation for multiple agents in simulation	Integrate task allocation, test and improve	Improve and test task allocation efficiency	Efficient task allocation for multiple agents

SVD Test Plan

Test #	Objective	Process	Requirements
1	Build multiple maps of test environment using different SLAM packages, evaluate map accuracy	Run SLAM package, which takes in odometry and 2D laser scan messages. Teleoperate the robot to explore and map the environment and repeat this for all packages. Run same localization algorithm with rosbag playback, except with each different map built with a SLAM package. Use map with lowest RMS error	M.F.1
2	Ensure that the robot is able to track the trajectory such that it is able to reach its required target location	Create the test path with a tape and place the robot at the start location. Give the command to start the motion and observe the robot as it follows the trajectory.	M.F.3

SVD Test Plan

Test #	Objective	Process	Requirements
3	To verify the ability of agents to localise in the given map and then navigate to different waypoints allocated by the server	Feed the map of the environment to the system, and the initial positions of the robots and let the robots localize in the environment. Run the task allocation node and let it allocate search tasks to different robots in the fleet and observe if the robots are able to navigate to required locations.	M.F.3
4	Ensure non interfering low cost task allocation for the multi agent system	Introduce the agents into the test area. Give 10 goal points to the entire system (for a system of 3 agents). Run the server + Nav stack on the system and observe if all goal points are reached.	M.F.2
5	To ensure that the path planned avoids collision with obstacles known in the map and other agents	Send different goal points to multiple robots. Observe the path being followed by the robots to go from its source point to the goal point. Repeat this by giving multiple goal points and observe for any collisions.	M.F.3

FVD Test Plan

Test #	Objective	Process	Requirements
1	Test the ability of system to detect and localize victims	Introduce 5 victims in the test area. Start the system and observe it navigating to various task locations. Once victim is encountered, verify if system is able to detect and localize it	M.F.0, M.P.0, M.P.1
2	Test the ability of the system to operate with a minimum of 3 agents in the fleet.	Start the search operation with 3 agents. Verify that all the agents are able to complete without collision as well as localize all victims encountered	M.P.5
3	Test the ability of system to perform with variable fleet size	Start the search operation with 3 agents. Let it continue for 10 seconds. Send a emergency stop command to one of the agent and verify if the other agents are able to complete all tasks. Also verify if this add/subtract of agents is successfully done via the GUI	M.F.4, M.N.2
4	Test the ability of system to visualize the current status on the GUI	Start the search operation with 3 agents and 5 victims. Observe if the location of all agents is visible on GUI. Also observe if all the victim positions are visible on the GUI at the end of search operation.	M.N.0

SVD Demo

Test location	4th Floor Wean Hall	
Procedure	• Agents will be placed at initial starting position within the operating area. • The user will give the agents an initial position estimate, along with a map of the environment. • The agents will be assigned tasks. • The agents will localize in the environment, and track the trajectory to complete the tasks. • The search is finished after every task is completed.	
Verification Criteria	System will have downtime of less than 5s for busy agents.	M.P.3
	System will update agent status for users at 1Hz frequency.	M.P.4
	Agents will finish searching environment 20% faster than the naive algorithm.	M.P.2



FVD Demo

Test location	4th Floor Wean
Procedure	• Agents will be placed at initial starting position within the operating area. • The user will give the agents an initial position estimate, along with a map of the environment. • The agents will be assigned tasks. • The agents will localize in the environment, and track the trajectory to complete the tasks. • Agents will move in a coordinated formation. • Agents will detect victims/fiducial markers and report their location to the user. • The map, agent status, and victim locations will be visualized on a GUI.
Verification Criterla	• System will detect 75% of victims in the environment • System will update agent status for user at 1Hz frequency.

Budget

The cost for this project is low. The project is largely software based, so very few physical components are needed. The Khepera robots are provided by our sponsor, so they do not add to the cost. Additionally, some of the sensors we are interested in are available in the MRSD inventory, so they will not add to the cost either. However, because we have many agents, most items must be ordered in large quantities. Note that table 11 shows Rough Order of Magnitude (ROM) costs.

Name	Description	Source	ROM Cost	Qty	Subtotal
Khepera IV	Robot platform for agents	Sponsor	0	10	0
Khepera LRFs	Laser Range Finder	Sponsor	0	5	0
Miscellaneous	Office Supplies	Purchased	-	-	145
				Total ($)	145 (~3%)

Risk Management

	Risk	Likeli-hood	Conse-quence	Mitigation Plan
1	Failed kin recognition on server due to mismatch of robot pose on server and actual location	2	4	• Turn off robot once the confidence covariance matrix goes below a certain threshold • Use a robot model size in proportion to the confidence covariance matrix of robot location for kin recognition
2	Agents sensors interfering with each other	4	4	• Test how sensors interact as soon as possible • Investigate sensor alternatives without interference • Design algorithms to rely on sensor without interference
3	Unable to procure the required sensors for localization on Kheperas	2	4	• Procuring lower quality sensors which might be available • Localize using camera imaging • Shift the localization task to Jackals (alternate robots)
4	Swarm algorithm failure due to bad data from faulty agents	1	3	• Robust swarm control algorithm • Rugged testing for edge case and failure • Simulate faulty agents in centralised control and discard
5	CPU usage overloading on the robots	2	4	• Offload computation to server as much as possible • ‘CPU compute requirement’ included as a performance criteria in all algorithm trade studies

Questions?

Backup Slides

System-level requirements: Desirable Requirements

D.F.0.	System shall map and search in unknown environment simultaneously during operation
D.F.1.	System shall detect and move obstacles during search (non-prehensile manipulation)
D.F.2.	System shall support flocking/formation control of multiple robots executing similar trajectories
D.F.3.	System shall account for victims needing simultaneous detection by multiple robots
D.F.4.	System shall account for victims needing detection by specific types of robots

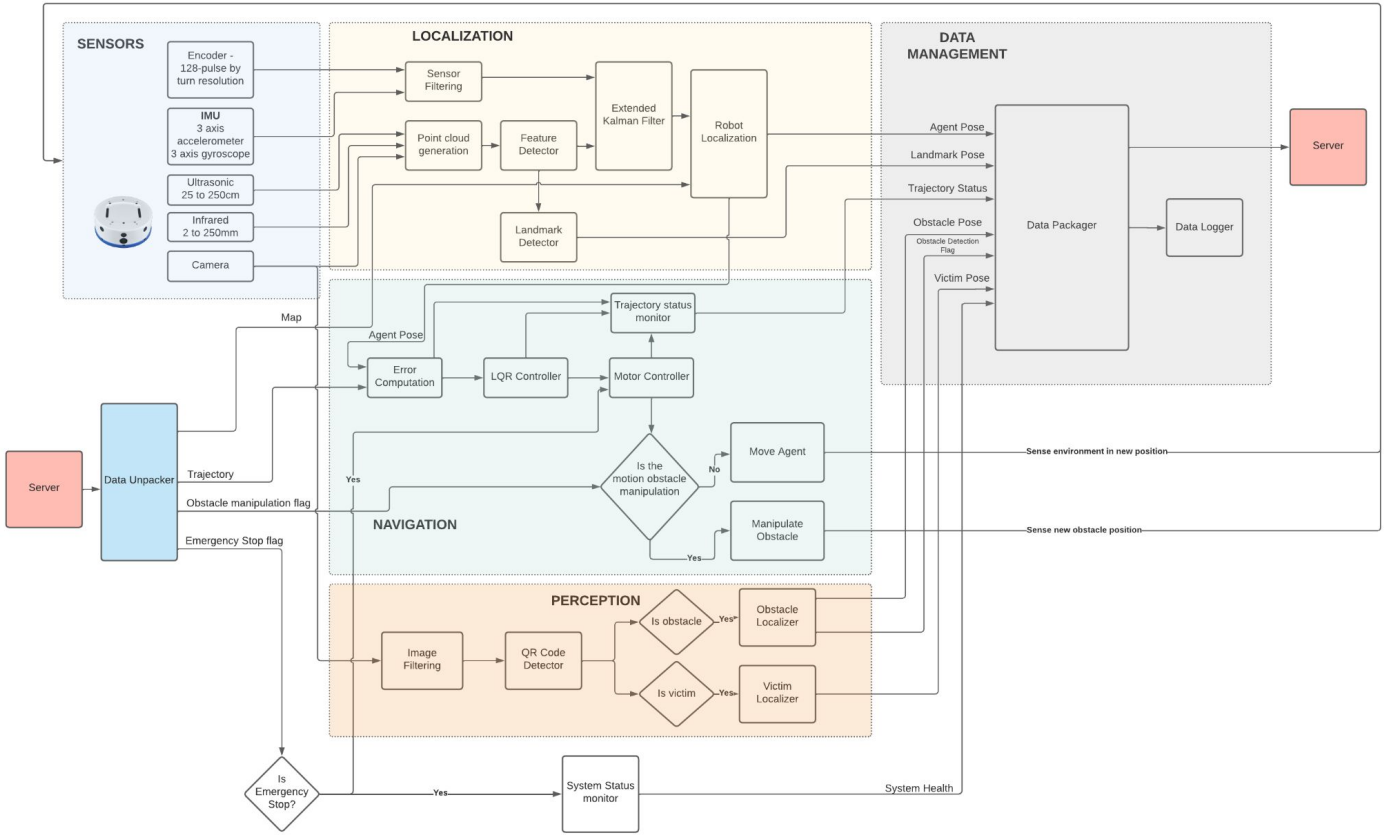
System-level requirements: Desirable Performance Requirements

D.P.0.	System will detect movable obstacles with 95% accuracy.
D.P.1.	System will successfully move obstacles of less than 15g, and 12.5x6.6x6.6cm.

System-level requirements: Desirable Non-Functional Requirements

D.N.0.	System shall visualize the locations of obstacles during operation
D.N.1.	System shall visualize victims needing multiple or special robots
D.N.2.	System shall support heterogenous robot search and control

Cyberphysical architecture : Agents old



Rubric

Preliminary Design Review Element	Weight
1. Project description	0.5
2. Use case	1
3. System-level requirements	1.5
4. Functional architecture	1
5. Cyberphysical architecture	1
6. System/subsystem descriptions	2.5
7. Current system status	2.5
8a. Project management: Work Breakdown Structure	1
8b. Project management: Schedule	0.8
8c. Project management: High-level test plan	2
8d. Project management: Budget	0.7
8e. Project management: Risk management	1.5
9a. Length	1
9b. Intelligibility (incl. format), flow, demeanor, audience connection	2
9c. Handling of Q & A	1
Total:	20